4.3. After the First Boot

If you activated the task "Debian desktop environment" without any explicit desktop choice (or with the "GNOME" choice), the computer will display the `gdm3` login manager.

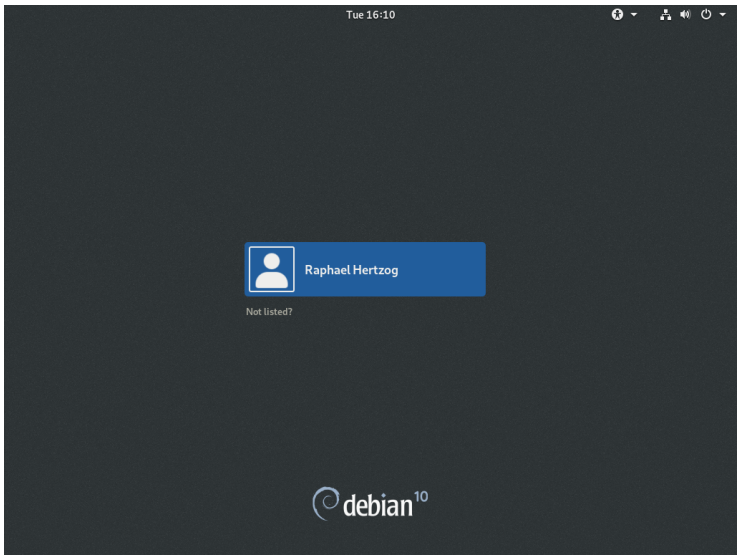


Figure 4.14 *First boot*

The user that has already been created can then log in and begin working immediately.

4.3.1. Installing Additional Software

The installed packages correspond to the profiles selected during installation, but not necessarily to the use that will actually be made of the machine. As such, you might want to use a package management tool to refine the selection of installed packages. The two most frequently used tools (which are installed if the "Debian desktop environment" profile was chosen) are `apt` (accessible from the command line) and `synaptic` ("Synaptic Package Manager" in the menus).

To facilitate the installation of coherent groups of programs, Debian creates "tasks" that are dedicated to specific uses (mail server, file server, etc.). You already had the opportunity to select them during installation, and you can access them again thanks to package management tools such as `aptitude` (the tasks are listed in a distinct section) and `synaptic` (through the menu Edit → Mark Packages by Task...).

Aptitude is an interface to APT in full-screen text mode. It allows the user to browse the list of available packages according to various categories (installed or not-installed packages, by task, by section, etc.), and to view all of the information available on each of them (dependencies, conflicts, description, etc.). Each package can be marked "install" (to be installed, + key) or "re-

move" (to be removed, - key). All of these operations will be conducted simultaneously once you've confirmed them by pressing the g key ("g" for "go!"). If you have forgotten some programs, no worries; you will be able to run `aptitude` again once the initial installation has been completed.

TIP

Debian thinks of speakers of non-English languages

Several tasks are dedicated to the localization of the system in other languages beyond English. They include translated documentation, dictionaries, and various other packages useful for speakers of different languages. The appropriate task is automatically selected if a non-English language was chosen during installation.

Of course, it is possible not to select any task to be installed. In this case, you can manually install the desired software with the `apt` or `aptitude` command (which are both accessible from the command line).

VOCABULARY

Package dependencies, conflicts

In the Debian packaging lingo, a "dependency" is another package necessary for the proper functioning of the package in question. Conversely, a "conflict" is a package that can not be installed side-by-side with another.

These concepts are discussed in greater detail in chapter 5, "Packaging System: Tools and Fundamental Principles" page 78.

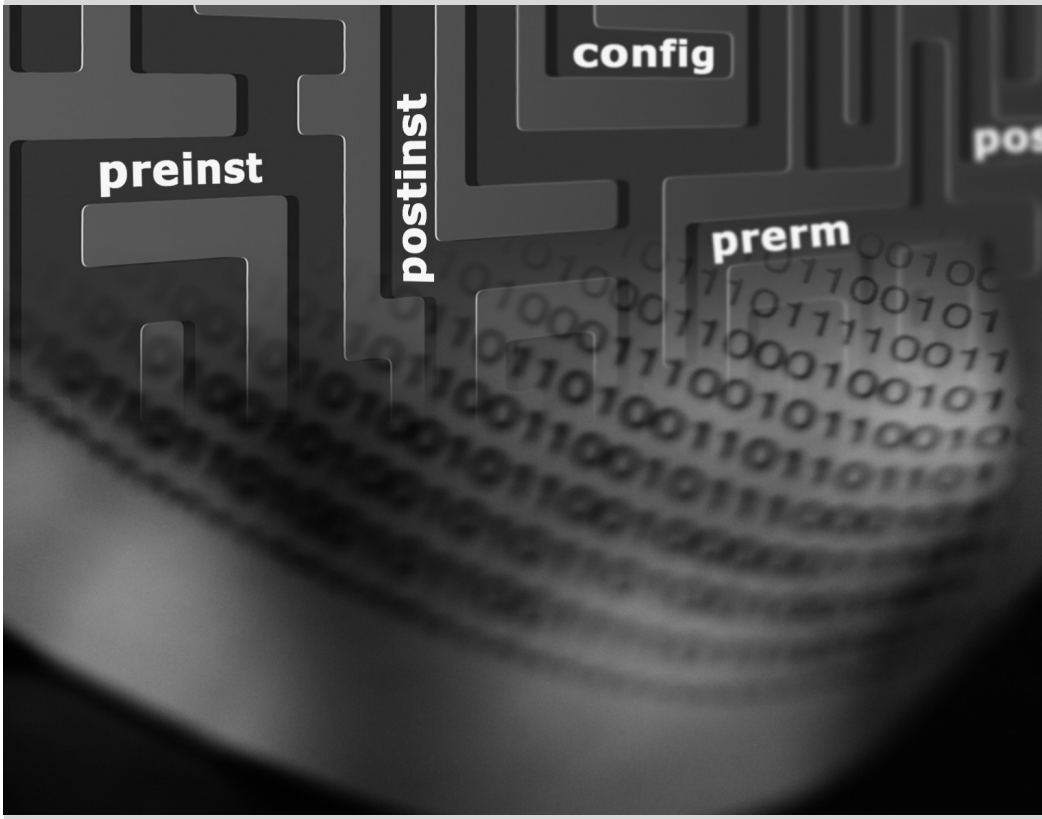
4.3.2. Upgrading the System

A first `apt upgrade` (a command used to automatically update installed programs) is generally required, especially for possible security updates issued since the release of the latest Debian stable version. These updates may involve some additional questions through `debconf`, the standard Debian configuration tool. For further information on these updates conducted by `apt`, please refer to section 6.2.3, "System Upgrade" page 120.



Keywords

Binary package
Source package
dpkg
deb
dependencies
conflict



Chapter

5

Packaging System: Tools and Fundamental Principles

Contents

Structure of a Binary Package 78 Package Meta-Information 80 Structure of a Source Package 90
Manipulating Packages with dpkg 94 Coexistence with Other Packaging Systems 103

As a Debian system administrator, you will routinely handle `.deb` *packages, since they contain consistent functional units (applications, documentation, etc.), whose installation and maintenance they facilitate. It is therefore a good idea to know what they are and how to use them.*

This chapter describes the structure and contents of "binary" and "source" packages. The former are files directly usable by `dpkg`, while the latter contain the source code, as well as instructions for building binary packages.

5.1. Structure of a Binary Package

The Debian package format is designed so that its content may be extracted on any Unix system that has the classic commands `ar`, `tar`, and `xz` or sometimes `gzip` or `bzip2`. This seemingly trivial property is important for portability and disaster recovery.

Imagine, for example, that you mistakenly deleted the `dpkg` program, and that you could thus no longer install Debian packages. `dpkg` being a Debian package itself, it would seem your system would be done for… Fortunately, you know the format of a package and can therefore download[1] the `.deb` file of the *dpkg* package and install it manually (see sidebar "dpkg, APT and ar" page 78). If by some misfortune one or more of the programs `ar`, `tar` or `gzip/xz/bzip2` have disappeared, you will only need to copy the missing program from another system (since each of these operates in a completely autonomous manner, without dependencies, a simple copy will suffice). If your system suffered some even more outrageous fortune, and even these don't work (maybe the deepest system libraries are missing?), you should try the static version of `busybox` (provided in the *busybox-static* package), which is even more self-contained, and provides subcommands such as `busybox ar`, `busybox tar` and `busybox xz`.

In case of a misfortune you better also have a backup of your system (see section 9.10, "Backup" page 227).

TOOLS

dpkg, APT and ar

`dpkg` is the program that handles `.deb` files (binary packages), notably extracting, analyzing, and unpacking them.

APT (the abbreviation of "Advanced Packaging Tool") is a group of programs that allows the execution of higher-level modifications to the system: installing or removing a package (while keeping dependencies satisfied), updating and upgrading the system, listing the available packages, etc.

As for the `ar` program, it allows handling files of the same name: `ar t` *`archive`* displays the list of files contained in such an archive, `ar x` *`archive`* extracts the files from the archive into the current working directory, `ar d` *`archive file`* deletes a file from the archive, etc. Its man page (`ar(1)`) documents all its other features. `ar` is a very rudimentary tool that a Unix administrator would only use on rare occasions, but admins routinely use `tar`, a more evolved archive and file management program. This is why it is easy to restore `dpkg` in the event of an erroneous deletion. You would only have to download the Debian package and extract the content from the `data.tar.xz` archive in the system's root (`/`):

```
# ar x dpkg_1.19.7_amd64.deb
# tar -C / -p -xJf data.tar.xz
```

[1] https://www.debian.org/distrib/packages#search_packages

BACK TO BASICS

Man page notation

It can be confusing for beginners to find references to "`ar(1)`" in the literature. This is generally a convenient means of referring to the man page entitled `ar` in section 1.

Sometimes this notation is also used to remove ambiguities, for example to distinguish between the `printf` command that can also be indicated by `printf(1)` and the `printf` function in the C programming language, which can also be referred to as `printf(3)`.

chapter 7, "Solving Problems and Finding Relevant Information" page 148 discusses manual pages in further detail (see section 7.1.1, "Manual Pages" page 148).

Have a look at the content of a `.deb` file:

```
$ ar t dpkg_1.19.7_amd64.deb
debian-binary
control.tar.gz
data.tar.xz
$ ar x dpkg_1.19.7_amd64.deb
$ ls
control.tar.gz  data.tar.xz  debian-binary  dpkg_1.19.7_amd64.deb
$ tar tJf data.tar.xz | head -n 16
./
./
./etc/
./etc/alternatives/
./etc/alternatives/README
./etc/cron.daily/
./etc/cron.daily/dpkg
./etc/dpkg/
./etc/dpkg/dpkg.cfg
./etc/dpkg/dpkg.cfg.d/
./etc/logrotate.d/
./etc/logrotate.d/alternatives
./etc/logrotate.d/dpkg
./sbin/
./sbin/start-stop-daemon
./usr/
./usr/bin/
$ tar tJf control.tar.xz
./
./conffiles
./control
./md5sums
./postinst
./postrm
$ cat debian-binary
2.0
```

As you can see, the `ar` archive of a Debian package is comprised of three files:

debian-binary This is a text file which simply indicates the version of the `.deb` file package format version. In Debian *Buster* it is still version 2.0.

control.tar.xz This archive file contains all of the available meta-information, like the name and version of the package as well as some scripts to run before, during or after (un-)installation of it. Some of the meta-information allows package management tools to determine if it is possible to install or uninstall it, for example according to the list of packages already on the machine, and if files shipped have been modified locally.

data.tar.xz, data.tar.bz2, data.tar.gz This archive contains all of the files to be extracted from the package; this is where the executable files, libraries, documentation, etc., are all stored. Packages may use different compression formats, in which case the file will be named differently for `xz`, `bzip2` or `gzip`.

5.2. Package Meta-Information

The Debian package is not only an archive of files intended for installation. It is part of a larger whole and describes its relationship with other Debian packages (requisites, dependencies, conflicts, suggestions). It also provides scripts that enable the execution of commands at different stages in the package's lifecycle (installation, upgrade, removal). These data are used by the package management tools but are not part of the packaged software; they are, within the package, what is called its "meta-information" (information about other information).

5.2.1. Description: the `control` File

This file uses a structure similar to email headers (as defined by RFC 2822) and is fully described in the Debian Policy and the manual pages `deb-control(5)` and `deb822(5)`.

➥ https://www.debian.org/doc/debian-policy/ch-controlfields.html

For example, for *apt*, the `control` file looks like the following:

```
$ apt-cache show apt
Package: apt
Version: 1.8.2
Installed-Size: 4064
Maintainer: APT Development Team <deity@lists.debian.org>
Architecture: amd64
Replaces: apt-transport-https (<< 1.5~alpha4~), apt-utils (<< 1.3~exp2~)
Provides: apt-transport-https (= 1.8.2)
Depends: adduser, gpgv | gpgv2 | gpgv1, debian-archive-keyring, libapt-pkg5.0 (>=
    ➥ 1.7.0~alpha3~), libc6 (>= 2.15), libgcc1 (>= 1:3.0), libgnutls30 (>= 3.6.6),
    ➥ libseccomp2 (>= 1.0.1), libstdc++6 (>= 5.2)
Recommends: ca-certificates
Suggests: apt-doc, aptitude | synaptic | wajig, dpkg-dev (>= 1.17.2), gnupg | gnupg2
    ➥ | gnupg1, powermgmt-base
```

```
Breaks: apt-transport-https (<< 1.5~alpha4~), apt-utils (<< 1.3~exp2~), aptitude (<<
    ➥ 0.8.10)
Description-en: commandline package manager
 This package provides commandline tools for searching and
 managing as well as querying information about packages
 as a low-level access to all features of the libapt-pkg library.
 .
 These include:
  * apt-get for retrieval of packages and information about them
    from authenticated sources and for installation, upgrade and
    removal of packages together with their dependencies
  * apt-cache for querying available information about installed
    as well as installable packages
  * apt-cdrom to use removable media as a source for packages
  * apt-config as an interface to the configuration settings
  * apt-key as an interface to manage authentication keys
Description-md5: 9fb97a88cb7383934ef963352b53b4a7
Tag: admin::package-management, devel::lang:ruby, hardware::storage,
 hardware::storage:cd, implemented-in::c++, implemented-in::perl,
 implemented-in::ruby, interface::commandline, network::client,
 protocol::ftp, protocol::http, protocol::ipv6, role::program,
 scope::application, scope::utility, suite::debian, use::downloading,
 use::organizing, use::playing, use::searching, works-with-format::html,
 works-with::audio, works-with::software:package, works-with::text
Section: admin
Priority: required
Filename: pool/main/a/apt/apt_1.8.2_amd64.deb
Size: 1418108
MD5sum: 0e80dedab6ec1e66a8f6c15f1925d2d3
SHA256: 80e9600822c4943106593ca5b0ec75d5aafa74c6130ba1071b013c42c507475e
```

BACK TO BASICS

RFC — Internet standards

RFC is the abbreviation of "Request For Comments". An RFC is generally a technical document that describes what will become an Internet standard. Before becoming standardized and frozen, these standards are submitted for public review (hence their name). The IETF (Internet Engineering Task Force) decides on the evolution of the status of these documents (proposed standard, draft standard, or standard).

RFC 2026 defines the process for standardization of Internet protocols.

➨ http://www.faqs.org/rfcs/rfc2026.html

Dependencies: the Depends Field

The dependencies are defined in the Depends field in the package header. It is a list of conditions to be met for the package to work correctly. This information is used by tools such as `apt` in order to install the required libraries, tools, drivers, etc. in appropriate versions fulfilling the dependencies of the package to be installed. For each dependency, it is possible to restrict the range of versions that meet that condition. In other words, it is possible to express the fact

that we need the package *libc6* in a version equal to or greater than "2.15" (written "`libc6 (>= 2.15)`"). Version comparison operators are as follows:

- <<: less than;
- <=: less than or equal to;
- =: equal to (note that "2.6.1" is not equal to "2.6.1-1");
- >=: greater than or equal to;
- >>: greater than.

In a list of conditions to be met, the comma serves as a separator. It must be interpreted as a logical "and". In conditions, the vertical bar ("|") expresses a logical "or" (it is an inclusive "or", not an exclusive "either/or"). Carrying greater priority than "and", it can be used as many times as necessary. Thus, the dependency "(A or B) and C" is written `A | B, C`. In contrast, the expression "A or (B and C)" should be written as "(A or B) and (A or C)", since the Depends field does not tolerate parentheses that change the order of priorities between the logical operators "or" and "and". It would thus be written `A | B, A | C`.

➨ https://www.debian.org/doc/debian-policy/#document-ch-relationships

The dependencies system is a good mechanism for guaranteeing the operation of a program, but it has another use with "meta-packages". These are empty packages that only describe dependencies. They facilitate the installation of a consistent group of programs preselected by the meta-package maintainer; as such, `apt install` *`meta-package`* will automatically install all of these programs using the meta-package's dependencies. The *gnome*, *kde-full* and *linux-image-amd64* packages are examples of meta-packages.

DEBIAN POLICY

Recommends, Suggests, and Enhances fields

The `Recommends` and `Suggests` fields describe dependencies that are not compulsory. The "recommended" dependencies, the most important, considerably improve the functionality offered by the package but are not indispensable to its operation. The "suggested" dependencies, of secondary importance, indicate that certain packages may complement and increase their respective utility, but it is perfectly reasonable to install one without the others.

You should always install the "recommended" packages, unless you know exactly why you do not need them. This is now also the default for APT unless configured otherwise. Conversely, it is not necessary to install "suggested" packages unless you know why you need them. The behavior of `apt` can be controlled by using the `APT::Install-Recommends` and `APT::Install-Suggests` configuration options or the corresponding command line options `--[no-]install-recommends` and `--[no-]install-suggests`.

The `Enhances` field also describes a suggestion, but in a different context. It is indeed located in the suggested package, and not in the package that benefits from the suggestion. Its interest lies in that it is possible to add a suggestion without having to modify the package that is concerned. Thus, all add-ons, plug-ins, and other extensions of a program can then appear in the list of suggestions related to the software. Although it has existed for several years, this last field is still largely ignored by programs such as `apt` or `synaptic`. Its purpose is for a suggestion made by the `Enhances` field to appear to the user in addition to the traditional suggestions — found in the `Suggests` field.

DEBIAN POLICY

Pre-Depends, a more demanding Depends

"Pre-dependencies", which are listed in the "`Pre-Depends`" field in the package headers, complete the normal dependencies; their syntax is identical. A normal dependency indicates that the package in question must be unpacked and configured before configuration of the package declaring the dependency. A pre-dependency stipulates that the package in question must be unpacked and configured before execution of the pre-installation script of the package declaring the pre-dependency, that is before its installation.

A pre-dependency is very demanding for `apt`, because it adds a strict constraint on the ordering of the packages to install. As such, pre-dependencies are discouraged unless absolutely necessary. It is even recommended to consult other developers on debian-devel@lists.debian.org before adding a pre-dependency. It is generally possible to find another solution as a work-around.

Conflicts: the Conflicts field

The Conflicts field indicates when a package cannot be installed simultaneously with another. The most common reasons for this are that both packages include a file of the same name and path, or provide the same service on the same TCP port, or would hinder each other's operation.

`dpkg` will refuse to install a package if it triggers a conflict with an already installed package, except if the new package specifies that it will "replace" the installed package, in which case `dpkg` will choose to replace the old package with the new one. `apt` always follows your instructions: if you choose to install a new package, it will automatically offer to uninstall the package that poses a problem.

Incompatibilities: the Breaks Field

The Breaks field has an effect similar to that of the Conflicts field, but with a special meaning. It signals that the installation of a package will "break" another package (or particular versions of it). In general, this incompatibility between two packages is transitory, and the Breaks relationship specifically refers to the incompatible versions.

`dpkg` will refuse to install a package that breaks an already installed package, and `apt` will try to resolve the problem by updating the package that would be broken to a newer version (which is assumed to be fixed and, thus, compatible again).

This type of situation may occur in the case of updates without backwards compatibility: this is the case if the new version no longer functions with the older version, and causes a malfunction in another program without making special provisions. The Breaks field prevents the user from running into these problems.

Provided Items: the Provides Field

This field introduces the very interesting concept of a "virtual package". It has many roles, but two are of particular importance. The first role consists in using a virtual package to associate a

generic service with it (the package "provides" the service). The second indicates that a package completely replaces another, and that for this purpose it can also satisfy the dependencies that the other would satisfy. It is thus possible to create a substitution package without having to use the same package name.

VOCABULARY

Meta-package and virtual package

It is essential to clearly distinguish meta-packages from virtual packages. The former are real packages (including real `.deb` files), whose only purpose is to express dependencies.

Virtual packages, however, do not exist physically; they are only a means of identifying real packages based on common, logical criteria (service provided, compatibility with a standard program or a pre-existing package, etc.).

Providing a "Service" Let us discuss the first case in greater detail with an example: all mail servers, such as *postfix* or *sendmail* are said to "provide" the *mail-transport-agent* virtual package. Thus, any package that needs this service to be functional (e.g. a mailing list manager, such as *smartlist* or *sympa*) simply states in its dependencies that it requires a *mail-transport-agent* instead of specifying a large yet incomplete list of possible solutions (e.g. `postfix | sendmail | exim4 | …`). Furthermore, it is useless to install two mail servers on the same machine, which is why each of these packages declares a conflict with the *mail-transport-agent* virtual package. A conflict between a package and itself is ignored by the system, but this technique will prohibit the installation of two mail servers side by side.

DEBIAN POLICY

List of virtual packages

For virtual packages to be useful, everyone must agree on their name. This is why they are standardized in the Debian Policy. The list includes among others *mail-transport-agent* for mail servers, *c-compiler* for C programming language compilers, *www-browser* for web browsers, *httpd* for web servers, *ftp-server* for FTP servers, *x-terminal-emulator* for terminal emulators in graphical mode (`xterm`), and *x-window-manager* for window managers.

The full list can be found on the Web.

➨ `http://www.debian.org/doc/packaging-manuals/virtual-package-names-list.txt`

Interchangeability with Another Package The Provides field is also interesting when the content of a package is included in a larger package. For example, the *libdigest-md5-perl* Perl module was an optional module in Perl 5.6, and has been integrated as standard in Perl 5.8 (and later versions, such as 5.28 present in *Buster*). As such, the package *perl* has since version 5.8 declared Provides: libdigest-md5-perl so that the dependencies on this package are met if the user has Perl 5.8 (or newer). The *libdigest-md5-perl* package itself has eventually been deleted, since it no longer had any purpose when old Perl versions were removed.

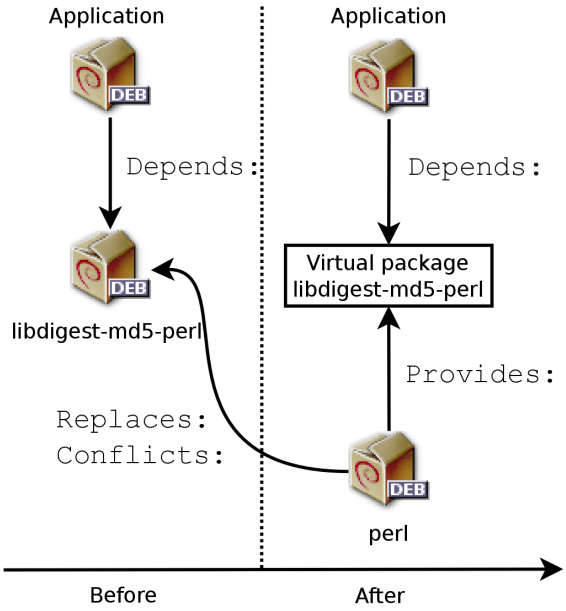


Figure 5.1 *Use of a Provides field in order to not break dependencies*

This feature is very useful, since it is never possible to anticipate the vagaries of development, and it is necessary to be able to adjust to renaming, and other automatic replacement, of obsolete software.

BACK TO BASICS

Perl, a programming language

Perl (Practical Extraction and Report Language) is a very popular programming language. It has many ready-to-use modules that cover a vast spectrum of applications, and that are distributed by the CPAN (Comprehensive Perl Archive Network) servers, an exhaustive network of Perl packages.

➨ `https://www.perl.org/`

➨ `https://www.cpan.org/`

Since it is an interpreted language, a program written in Perl does not require compilation prior to execution. This is why they are called "Perl scripts".

Past Limitations Virtual packages used to suffer from some limitations, the most significant of which was the absence of a version number. To return to the previous example, a dependency such as Depends: libdigest-md5-perl (>= 1.6), despite the presence of Perl 5.10, would never be considered as satisfied by the packaging system — while in fact it most likely is satisfied. Unaware of this, the package system chose the least risky option, assuming that the versions do not match.

This limitation has been lifted in *dpkg* 1.17.11, and is no longer relevant. Packages can assign a version to the virtual packages they provide with a dependency such as Provides: libdigest-md5-perl (= 1.8).

Replacing Files: The Replaces Field

The Replaces field indicates that the package contains files that are also present in another package, but that the package is legitimately entitled to replace them. Without this specification, `dpkg` fails, stating that it can not overwrite the files of another package (technically, it is possible to force it to do so with the --force-overwrite option, but that is not considered standard operation). This allows identification of potential problems and requires the maintainer to study the matter prior to choosing whether to add such a field.

The use of this field is justified when package names change or when a package is included in another. This also happens when the maintainer decides to distribute files differently among various binary packages produced from the same source package: a replaced file no longer belongs to the old package, but only to the new one.

If all of the files in an installed package have been replaced, the package is considered to be removed. Finally, this field also encourages `dpkg` to remove the replaced package where there is a conflict.

GOING FURTHER

The Tag field

In the *apt* example above, we can see the presence of a field that we have not yet described, the Tag field. This field does not describe a relationship between packages, but is simply a way of categorizing a package in a thematic taxonomy. This classification of packages according to several criteria (type of interface, programming language, domain of application, etc.) has been available for a long time. Despite this, not all packages have accurate tags and it is not yet integrated in all Debian tools; `aptitude` displays these tags, and allows them to be used as search criteria. For those who are repelled by `aptitude`'s search criteria, the following website allows navigation of the tag database:

➨ https://wiki.debian.org/Debtags

5.2.2. Configuration Scripts

In addition to the `control` file, the `control.tar.gz` archive for each Debian package may contain a number of scripts, called by `dpkg` at different stages in the processing of a package. The Debian Policy describes the possible cases in detail[2], specifying the scripts called and the arguments that they receive. These sequences may be complicated, since if one of the scripts fails, `dpkg` will try to return to a satisfactory state by canceling the installation or removal in progress (insofar as it is possible).

GOING FURTHER

dpkg's database

All of the configuration scripts for installed packages are stored in the `/var/lib/dpkg/info/` directory, in the form of a file prefixed with the package's name. This directory also includes a file with the `.list` extension for each package, containing the list of files that belong to that package.

The `/var/lib/dpkg/status` file contains a series of data blocks (in the format of the famous mail headers, RFC 2822) describing the status of each package. The information from the `control` file of the installed packages is also replicated there.

[2] https://www.debian.org/doc/debian-policy/ch-maintainerscripts.html

In general, the `preinst` script is executed prior to installation of the package, while `postinst` follows it. Likewise, `prerm` is invoked before removal of a package and `postrm` afterwards. An update of a package is equivalent to removal of the previous version and installation of the new one. It is not possible to describe in detail all the possible scenarios here, but we will discuss the most common two: an installation/update and a removal.

CAUTION

Symbolic names of the scripts

The sequences described in this section call configuration scripts by specific names, such as `old-prerm` or `new-postinst`. They are, respectively, the `prerm` script contained in the old version of the package (installed before the update) and the `postinst` script contained in the new version (installed by the update).

TIP

State diagrams

Manoj Srivastava and Margarita Manterola made the following diagrams explaining how the configuration scripts are called by `dpkg`.

➨ `https://people.debian.org/~srivasta/MaintainerScripts.html`

➨ `https://www.debian.org/doc/debian-policy/ap-flowcharts.html`

Installation and Upgrade

Here is what happens during an installation (or an update):

1. For an update, `dpkg` calls the `old-prerm upgrade` *`new-version`*.
2. Still for an update, `dpkg` then executes `new-preinst upgrade` *`old-version`*; for a first installation, it executes `new-preinst install`. It may add the old version in the last parameter, if the package has already been installed and removed since (but not purged, the configuration files having been retained).
3. The new package files are then unpacked. If a file already exists, it is replaced, but a backup copy is temporarily made.
4. For an update, `dpkg` executes `old-postrm upgrade` *`new-version`*.
5. `dpkg` updates all of the internal data (file list, configuration scripts, etc.) and removes the backups of the replaced files. This is the point of no return: `dpkg` no longer has access to all of the elements necessary to return to the previous state.
6. `dpkg` will update the configuration files, asking the user to decide if it is unable to automatically manage this task. The details of this procedure are discussed in section 5.2.3, "Checksums, List of Configuration Files" page 89.
7. Finally, `dpkg` configures the package by executing `new-postinst configure` *`last-version-configured`*.

Package Removal

Here is what happens during a package removal:

1. `dpkg` calls `prerm remove`.
2. `dpkg` removes all of the package's files, with the exception of the configuration files and configuration scripts.
3. `dpkg` executes `postrm remove`. All of the configuration scripts, except `postrm`, are removed. If the user has not used the "purge" option, the process stops here.
4. For a complete purge of the package (command issued with `dpkg --purge` or `dpkg -P`), the configuration files are also deleted, as well as a certain number of copies (`*.dpkg-tmp`, `*.dpkg-old`, `*.dpkg-new`) and temporary files; `dpkg` then executes `postrm purge`.

VOCABULARY

Purge, a complete removal

When a Debian package is removed, the configuration files are retained in order to facilitate possible re-installation. Likewise, the data generated by a daemon (such as the content of an LDAP server directory, or the content of a database for an SQL server) are usually retained.

To remove all data associated with a package, it is necessary to "purge" the package with the command, `dpkg -P` *package*, `apt-get remove --purge` *package* or `aptitude purge` *package*.

Given the definitive nature of such data removals, a purge should not be taken lightly.

The four scripts detailed above are complemented by a `config` script, provided by packages using `debconf` to acquire information from the user for configuration. During installation, this script defines in detail the questions asked by `debconf`. The responses are recorded in the `debconf` database for future reference. The script is generally executed by `apt` prior to installing packages one by one in order to group all the questions and ask them all to the user at the beginning of the process. The pre- and post-installation scripts can then use this information to operate according to the user's wishes.

TOOL

`debconf`

`debconf` was created to resolve a recurring problem in Debian. All Debian packages unable to function without a minimum of configuration used to ask questions with calls to the `echo` and `read` commands in `postinst` shell scripts (and other similar scripts). But this also means that during a large installation or update the user must stay with their computer to respond to various questions that may arise at any time. These manual interactions have now been almost entirely dispensed with, thanks to the `debconf` tool.

`debconf` has many interesting features: it requires the developer to specify user interaction; it allows localization of all the strings displayed to users (all translations are stored in the `templates` file describing the interactions); it has different frontends to display the questions to the user (text mode, graphical mode, non-interactive); and it allows creation of a central database of responses to share the same configuration with several computers… but the most important is that it is now possible to present all of the questions in a row to the user, prior to starting a long installation or update process. The user can go about their business while the system handles the installation on its own, without having to stay there staring at the screen waiting for questions.

5.2.3. Checksums, List of Configuration Files

In addition to the maintainer scripts and control data already mentioned in the previous sections, the `control.tar.gz` archive of a Debian package may contain other interesting files. The first, `md5sums`, contains the MD5 checksums for all of the package's files. Its main advantage is that it allows `dpkg --verify` (which we will study in section 14.3.4.1, "Auditing Packages with `dpkg --verify`" page 413) and `debsums` (from the package of the same name; see section 14.3.4.2, "Auditing Packages: `debsums` and its Limits" page 414) to check if these files have been modified since their installation. Note that when this file doesn't exist, `dpkg` will generate it dynamically at installation time (and store it in the dpkg database just like other control files).

`conffiles` lists package files that must be handled as configuration files (see also `deb-conffiles(5)`). Configuration files can be modified by the administrator, and `dpkg` will try to preserve those changes during a package update.

In effect, in this situation, `dpkg` behaves as intelligently as possible: if the standard configuration file has not changed between the two versions, it does nothing. If, however, the file has changed, it will try to update this file. Two cases are possible: either the administrator has not touched this configuration file, in which case `dpkg` automatically installs the new version; or the file has been modified, in which case `dpkg` asks the administrator which version they wish to use (the old one with modifications, or the new one provided with the package). To assist in making this decision, `dpkg` offers to display a "`diff`" that shows the difference between the two versions. If the user chooses to retain the old version, the new one will be stored in the same location in a file with the `.dpkg-dist` suffix. If the user chooses the new version, the old one is retained in a file with the `.dpkg-old` suffix. Another available action consists of momentarily interrupting `dpkg` to edit the file and attempt to re-instate the relevant modifications (previously identified with `diff`).

GOING FURTHER

Force dpkg to ask configuration file questions

The `--force-confask` option requires dpkg to display the questions about the configuration files, even in cases where they would not normally be necessary. Thus, when reinstalling a package with this option, `dpkg` will ask the questions again for all of the configuration files modified or deleted by the administrator. This is very convenient, especially for reinstalling the original configuration file if it has been deleted and no other copy is available: a normal re-installation won't work, because `dpkg` considers removal as a form of legitimate modification, and, thus, doesn't install the desired configuration file.

GOING FURTHER

Avoiding the configuration file questions

`dpkg` handles configuration file updates, but, while doing so, regularly interrupts its work to ask for input from the administrator. This makes it less than enjoyable for those who wish to run updates in a non-interactive manner. This is why this program offers options that allow the system to respond automatically according to the same logic: `--force-confold` retains the old version of the file; `--force-confnew` will use the new version of the file (these choices are respected, even if the file has not been changed by the administrator, which only rarely has the desired effect). Adding the `--force-confdef` option tells `dpkg` to decide by itself when possible (in other words, when the original configuration file has not been touched), and only uses `--force-confnew` or `--force-confold` for other cases.

These options apply to dpkg and are explained in detail in dpkg(1) or dpkg --force-help, but most of the time the administrator will work directly with the aptitude or apt programs. It is, thus, necessary to know the syntax used to indicate the options to pass to the dpkg command (their command line interfaces are very similar).

```
# apt -o DPkg::options::="--force-confdef" -o DPkg::options
    ➥ ::="--force-confold" full-upgrade
```

These options can be stored directly in apt's configuration. To do so, simply write the following line in the /etc/apt/apt.conf.d/local file:

```
DPkg::options { "--force-confdef"; "--force-confold"; }
```

Including this option in the configuration file means that it will also be used in a graphical interface such as aptitude.

5.3. Structure of a Source Package

5.3.1. Format

A source package is usually comprised of three files, a .dsc, a .orig.tar.gz, and a .debian.tar.xz (or .diff.gz). They allow creation of binary packages (.deb files described above) from the source code files of the program, which are written in a programming language.

The .dsc (Debian Source Control) file is a short text file containing an RFC 2822 header (just like the control file studied in section 5.2.1, "Description: the control File" page 80) which describes the source package and indicates which other files are part thereof. It is signed by its maintainer, which guarantees authenticity. See section 6.6, "Checking Package Authenticity" page 132 for further details on this subject.

Example 5.1 *A .dsc file*

```
-----BEGIN PGP SIGNED MESSAGE-----
Hash: SHA512

Format: 3.0 (quilt)
Source: zim
Binary: zim
Architecture: all
Version: 0.68-1
Maintainer: Zim Package Maintainers <zim@packages.debian.org>
Uploaders: Raphaël Hertzog <hertzog@debian.org>
Homepage: http://zim-wiki.org
Standards-Version: 4.1.3
Vcs-Browser: https://salsa.debian.org/debian/zim
Vcs-Git: https://salsa.debian.org/debian/zim.git
```

```
Build-Depends: debhelper (>= 11), xdg-utils, python (>= 2.6.6-3~), libgtk2.0-0 (>=
    ➥ 2.6), python-gtk2, python-xdg, dh-python
Package-List:
 zim deb x11 optional arch=all
Checksums-Sha1:
 a3b50aa8e44126cc7edd2c1912adf9820f50ad58 2044224 zim_0.68.orig.tar.gz
 4e13b37625789334da2d95b93e51e41ffd3b6b01 9300 zim_0.68-1.debian.tar.xz
Checksums-Sha256:
 d91518e010f6a6e951a75314138b5545a4c51151fc99f513aa7768a18858df15 2044224 zim_0.68.
     ➥ orig.tar.gz
 23f4ddc69af74509932acc3b5f0d4cd2af943016e4fd5740b9d98ec4d49fd8c2 9300 zim_0.68-1.
     ➥ debian.tar.xz
Files:
 336041a16687abb66fd9f604b98407e8 2044224 zim_0.68.orig.tar.gz
 1714f67b35ab69e709849ad707206ca8 9300 zim_0.68-1.debian.tar.xz

-----BEGIN PGP SIGNATURE-----
Comment: Signed by Raphael Hertzog

iQEzBAEBCgAdFiEE1823g1EQnhJ1LsbSA4gdq+vCmrkFAlqyOxkACgkQA4gdq+vC
mrnCqAf/Ww9wg97VragtVhSFvehoVoJ0ZhoqNaSuCP/W1Fuf+P0YklzL2BlkVRXW
X23c8Qs1v6VE2iRY3mEkdWwgBs1QwF0MX7H1jjQfPHCynGHKlH5dfo5fqLizgCeu
c9Pug3ZisjF90CgsseO7SVDqHVmO6QsfAaGWpHAw92HDz/xwjrS/4Ejntqjy0b+r
Gmw2AZuBdhp+7C6p7In/Gg6DHPBLQGMLCKypoZKQdl+L0fWjjeykOzMIbjry2sRH
H0J4FLVGAGumh3zIZlm/t3ehGfP9Dg8FvzMaCNsf8OtYCSAEutrQEDBaskcTSIpq
L0GQhKlViDuu8gzsqm7efPEhPcsF1A==
=6jGR
-----END PGP SIGNATURE-----
```

Note that the source package also has dependencies (Build-Depends) completely distinct from those of binary packages, since they indicate tools required to compile the software in question and construct its binary package.

CAUTION

Distinct namespaces

It is important to note here that there is no required correspondence between the name of the source package and that of the binary package(s) that it generates. It is easy enough to understand if you know that each source package may generate several binary packages. This is why the `.dsc` file has the `Source` and `Binary` fields to explicitly name the source package and store the list of binary packages that it generates.

CULTURE

Why divide into several packages

Quite frequently, a source package (for a given software) can generate several binary packages. The split is justified by the possibility to use (parts of) the software in different contexts. Consider a shared library, it may be installed to make an application work (for example, *libc6*), or it can be installed to develop a new program (*libc6-dev* will then be the correct package). We find the same logic for client/server services where we want to install the server part on one machine and the client part on others (this is the case, for example, of *openssh-server* and *openssh-client*).

Just as frequently, the documentation is provided in a dedicated package: the user may install it independently from the software, and may at any time choose to remove it to save disk space. Additionally, this also saves disk space on the Debian mirrors, since the documentation package will be shared amongst all of the architectures (instead of having the documentation duplicated in the packages for each architecture).

PERSPECTIVE

Different source package formats

Originally there was only one source package format. This is the `1.0` format, which associates an `.orig.tar.gz` archive to a `.diff.gz` "debianization" patch (there is also a variant, consisting of a single `.tar.gz` archive, which is automatically used if no `.orig.tar.gz` is available).

Since Debian 6 *Squeeze*, Debian developers have the option to use new formats that correct many problems of the historical format. Format `3.0 (quilt)` can combine multiple upstream archives in the same source package: in addition to the usual `.orig.tar.gz`, supplementary `.orig-`*`component`*`.tar.gz` archives can be included. This is useful with software that is distributed in several upstream components but for which a single source package is desired. These archives can also be compressed with `xz` rather than `gzip`, which saves disk space and network resources. Finally, the monolithic patch, `.diff.gz` is replaced by a `.debian.tar.xz` archive containing the compiling instructions and a set of upstream patches contributed by the package maintainer. These last are recorded in a format compatible with `quilt` — a tool that facilitates the management of a series of patches.

The `.orig.tar.gz` file is an archive containing the source code as provided by the original developer. Debian package maintainers are asked to not modify this archive in order to be able to easily check the origin and integrity of the file (by simple comparison with a checksum) and to respect the wishes of some authors.

The `.debian.tar.xz` contains all of the modifications made by the Debian maintainer, especially the addition of a `debian` directory containing the instructions to execute to construct a Debian package.

TOOL

Decompressing a source package

If you have a source package, you can use the `dpkg-source` command (from the *dpkg-dev* package) to decompress it:

```
$ dpkg-source -x zim_0.68-1.dsc
dpkg-source: info: extracting zim in zim-0.68
dpkg-source: info: unpacking zim_0.68.orig.tar.gz
dpkg-source: info: unpacking zim_0.68-1.debian.tar.xz
```

You can also use `apt` to download a source package and unpack it right away. It requires that the appropriate `deb-src` lines be present in the `/etc/apt/sources.list` file, however (for further details, see section 6.1, "Filling in the `sources.list` File" page 108). These are used to list the "sources" of source packages (meaning the servers on which a group of source packages are hosted).

```
$ apt source package
Reading package lists... Done
Selected version '0.68-1' (stable) for zim
```

```
NOTICE: 'zim' packaging is maintained in the 'Git' version
    ➥ control system at:
https://salsa.debian.org/debian/zim.git
Please use:
git clone https://salsa.debian.org/debian/zim.git
to retrieve the latest (possibly unreleased) updates to the
    ➥  package.
Need to get 2055 kB of source archives.
Get:1 https://cdn-aws.deb.debian.org/debian stable/main zim
    ➥  0.68-1 (dsc) [1586 B]
Get:2 https://cdn-aws.deb.debian.org/debian stable/main zim
    ➥  0.68-1 (tar) [2044 kB]
Get:3 https://cdn-aws.deb.debian.org/debian stable/main zim
    ➥  0.68-1 (diff) [9300 B]
Fetched 2055 kB in 1s (3356 kB/s)
dpkg-source: info: extracting zim in zim-0.68
dpkg-source: info: unpacking zim_0.68.orig.tar.gz
dpkg-source: info: unpacking zim_0.68-1.debian.tar.xz
```

5.3.2. Usage within Debian

The source package is the foundation of everything in Debian. All Debian packages come from a source package, and each modification in a Debian package is the consequence of a modification made to the source package. The Debian maintainers work with the source package, knowing, however, the consequences of their actions on the binary packages. The fruits of their labors are thus found in the source packages available from Debian: you can easily go back to them and everything stems from them.

When a new version of a package (source package and one or more binary packages) arrives on the Debian server, the source package is the most important. Indeed, it will then be used by a network of machines of different architectures for compilation on the various architectures supported by Debian. The fact that the developer also sends one or more binary packages for a given architecture (usually i386 or amd64) is relatively unimportant, since these could just as well have been automatically generated.

➥ https://buildd.debian.org/

GOING FURTHER

Source only maintainer uploads

Right after the release of Debian 10 *Buster* the Release Team announced that maintainer binary uploads will no longer be accepted for `main` and all binary packages in this component will be built automatically from mandatory source-only uploads.

5.4. Manipulating Packages with dpkg

dpkg is the base command for handling Debian packages on the system. If you have .deb packages, it is dpkg that allows installation or analysis of their contents. But this program only has a partial view of the Debian universe: it knows what is installed on the system, and whatever it is given on the command line, but knows nothing of the other available packages. As such, it will fail if a dependency is not met. Tools such as apt and aptitude, on the contrary, will create a list of dependencies to install everything as automatically as possible.

NOTE **dpkg or apt?**

dpkg should be seen as a system tool (backend), and apt as a tool closer to the user, which overcomes the limitations of the former. These tools work together, each one with its particularities, suited to specific tasks.

5.4.1. Installing Packages

dpkg is, above all, the tool for installing an already available Debian package (because it does not download anything). To do this, we use its -i or --install option.

Example 5.2 *Installation of a package with dpkg*

```
# dpkg -i man-db_2.8.5-2_amd64.deb
(Reading database ... 14913 files and directories currently installed.)
Preparing to unpack .../man-db_2.8.5-2_amd64.deb ...
Unpacking man-db (2.8.5-2) over (2.8.5-2) ...
Setting up man-db (2.8.5-2) ...
Updating database of manual pages ...
Processing triggers for mime-support (3.62) ...
```

We can see the different steps performed by dpkg; we know, thus, at what point any error may have occurred. The installation can also be effected in two stages: first unpacking, then configuration. apt takes advantage of this, limiting the number of calls to dpkg (since each call is costly, due to loading of the database in memory, especially the list of already installed files).

Example 5.3 *Separate unpacking and configuration*

```
# dpkg --unpack man-db_2.8.5-2_amd64.deb
(Reading database ... 14937 files and directories currently installed.)
Preparing to unpack man-db_2.8.5-2_amd64.deb ...
Unpacking man-db (2.8.5-2) over (2.8.5-2) ...
Processing triggers for mime-support (3.62) ...
# dpkg --configure man-db
```

```
Setting up man-db (2.8.5-2) ...
Updating database of manual pages ...
```

Sometimes `dpkg` will fail to install a package and return an error; if the user orders it to ignore this, it will only issue a warning; it is for this reason that we have the different --force-* options. The `dpkg --force-help` command, or documentation of this command, will give a complete list of these options. The most frequent error, which you are bound to encounter sooner or later, is a file collision. When a package contains a file that is already installed by another package, `dpkg` will refuse to install it. The following messages will then appear:

```
Unpacking libgdm (from .../libgdm_3.8.3-2_amd64.deb) ...
dpkg: error processing /var/cache/apt/archives/libgdm_3.8.3-2_amd64.deb (--unpack):
 trying to overwrite '/usr/bin/gdmflexiserver', which is also in package gdm3 3.4.1-9
```

In this case, if you think that replacing this file is not a significant risk to the stability of your system (which is usually the case), you can use the option --force-overwrite, which tells `dpkg` to ignore this error and overwrite the file.

While there are many available --force-* options, only --force-overwrite is likely to be used regularly. These options only exist for exceptional situations, and it is better to leave them alone as much as possible in order to respect the rules imposed by the packaging mechanism. Do not forget, these rules ensure the consistency and stability of your system.

CAUTION

Effective use of `--force-*`

If you are not careful, the use of an option `--force-*` can lead to a system where the APT family of commands will refuse to function. In effect, some of these options allow installation of a package when a dependency is not met, or when there is a conflict. The result is an inconsistent system from the point of view of dependencies, and the APT commands will refuse to execute any action except those that will bring the system back to a consistent state (this often consists of installing the missing dependency or removing a problematic package). This often results in a message like this one, obtained after installing a new version of *rdesktop* while ignoring its dependency on a newer version of the *libc6*:

```
# apt full-upgrade
[...]
You might want to run 'apt-get -f install' to correct these
    ➥ .
The following packages have unmet dependencies:
  rdesktop: Depends: libc6 (>= 2.5) but 2.3.6.ds1-13etch7
      ➥ is installed
E: Unmet dependencies. Try using -f.
```

A courageous administrator who is certain of the correctness of their analysis may choose to ignore a dependency or conflict and use the corresponding `--force-*` option. In this case, if they want to be able to continue to use `apt` or `aptitude`, they must edit `/var/lib/dpkg/status` to delete/modify the dependency, or conflict, that they chose to override.

This manipulation is an ugly hack, and should never be used, except in the most extreme case of necessity. Quite frequently, a more fitting solution is to recompile the

package that is causing the problem (see section 15.1, "Rebuilding a Package from its Sources" page 448) or use a new version (potentially corrected) from a repository such as the `stable-backports` one (see section 6.1.2.4, "Stable Backports" page 112).

5.4.2. Package Removal

Invoking `dpkg` with the -r or --remove option, followed by the name of a package, removes that package. This removal is, however, not complete: all of the configuration files, maintainer scripts, log files (system logs) and other user data handled by the package remain. That way disabling the program is easily done by uninstalling it, and it is still possible to quickly reinstall it with the same configuration. To completely remove everything associated with a package, use the -P or --purge option, followed by the package name.

Example 5.4 *Removal and purge of the* debian-cd *package*

```
# dpkg -r debian-cd
(Reading database ... 15915 files and directories currently installed.)
Removing debian-cd (3.1.25) ...
# dpkg -P debian-cd
(Reading database ... 15394 files and directories currently installed.)
Purging configuration files for debian-cd (3.1.25) ...
```

5.4.3. Querying `dpkg`'s Database and Inspecting `.deb` Files

BACK TO BASICS

Option syntax

Most options are available in a "long" version (one or more relevant words, preceded by a double dash) and a "short" version (a single letter, often the initial of one word from the long version, and preceded by a single dash). This convention is so common that it is a POSIX standard.

Before concluding this section, we will study `dpkg` options that query the internal database in order to obtain information. Giving first the long options and then corresponding short options (that will evidently take the same possible arguments) we cite --listfiles *package* (or -L), which lists the files installed by this package; --search *file* (or -S), which finds the package(s) containing the file; --status *package* (or -s), which displays the headers of an installed package; --list (or -l), which displays the list of packages known to the system and their installation status; --contents *file.deb* (or -c), which lists the files in the Debian package specified; --info *file.deb* (or -I), which displays the headers of this Debian package.

CAUTION

`dpkg --search` and merged `/usr`

For various reasons[3], Debian now installs by default a few top-level directories as symlinks to their counterparts below `/usr`. For instance, `/bin`, `/sbin` and `/lib` are now symlinks to, respectively, `/usr/bin`, `/usr/sbin` and `/usr/lib`.

While this does provide desirable benefits, it can also be a source of confusion. For example, when you query `dpkg` which package is owning a given file, it will only be able to answer when you ask for its original path:

```
$ dpkg --search /bin/mount
mount: /bin/mount
$ dpkg --search /usr/bin/mount
dpkg-query: no path found matching pattern /usr/bin/mount
$ dpkg --search /bin/apt
dpkg-query: no path found matching pattern /bin/apt
$ dpkg --search /usr/bin/apt
apt: /usr/bin/apt
```

Example 5.5 *Various queries with dpkg*

```
$ dpkg -L base-passwd
/.
/usr
/usr/sbin
/usr/sbin/update-passwd
/usr/share
/usr/share/base-passwd
/usr/share/base-passwd/group.master
/usr/share/base-passwd/passwd.master
/usr/share/doc
/usr/share/doc/base-passwd
/usr/share/doc/base-passwd/README
/usr/share/doc/base-passwd/changelog.gz
/usr/share/doc/base-passwd/copyright
/usr/share/doc/base-passwd/users-and-groups.html
/usr/share/doc/base-passwd/users-and-groups.txt.gz
/usr/share/doc-base
/usr/share/doc-base/users-and-groups
/usr/share/lintian
/usr/share/lintian/overrides
/usr/share/lintian/overrides/base-passwd
/usr/share/man
/usr/share/man/de
/usr/share/man/de/man8
/usr/share/man/de/man8/update-passwd.8.gz
/usr/share/man/es
/usr/share/man/es/man8
```

[3] https://www.freedesktop.org/wiki/Software/systemd/TheCaseForTheUsrMerge/

```
/usr/share/man/es/man8/update-passwd.8.gz
/usr/share/man/fr
/usr/share/man/fr/man8
/usr/share/man/fr/man8/update-passwd.8.gz
/usr/share/man/ja
/usr/share/man/ja/man8
/usr/share/man/ja/man8/update-passwd.8.gz
/usr/share/man/man8
/usr/share/man/man8/update-passwd.8.gz
/usr/share/man/pl
/usr/share/man/pl/man8
/usr/share/man/pl/man8/update-passwd.8.gz
/usr/share/man/ru
/usr/share/man/ru/man8
/usr/share/man/ru/man8/update-passwd.8.gz
$ dpkg -S /bin/date
coreutils: /bin/date
$ dpkg -s coreutils
Package: coreutils
Essential: yes
Status: install ok installed
Priority: required
Section: utils
Installed-Size: 15719
Maintainer: Michael Stone <mstone@debian.org>
Architecture: amd64
Multi-Arch: foreign
Version: 8.30-3
Pre-Depends: libacl1 (>= 2.2.23), libattr1 (>= 1:2.4.44), libc6 (>= 2.28),
    ➥ libselinux1 (>= 2.1.13)
Description: GNU core utilities
 This package contains the basic file, shell and text manipulation
 utilities which are expected to exist on every operating system.
 .
 Specifically, this package includes:
 arch base64 basename cat chcon chgrp chmod chown chroot cksum comm cp
 csplit cut date dd df dir dircolors dirname du echo env expand expr
 factor false flock fmt fold groups head hostid id install join link ln
 logname ls md5sum mkdir mkfifo mknod mktemp mv nice nl nohup nproc numfmt
 od paste pathchk pinky pr printenv printf ptx pwd readlink realpath rm
 rmdir runcon sha*sum seq shred sleep sort split stat stty sum sync tac
 tail tee test timeout touch tr true truncate tsort tty uname unexpand
 uniq unlink users vdir wc who whoami yes
Homepage: http://gnu.org/software/coreutils
$ dpkg -l 'b*'
Desired=Unknown/Install/Remove/Purge/Hold
| Status=Not/Inst/Conf-files/Unpacked/halF-conf/Half-inst/trig-aWait/Trig-pend
|/ Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
||/ Name                      Version             Architecture Description
```

```
+++-==================-===============-===============-=================================
    ➥
un  backupninja          <none>          <none>          (no description
    ➥ available)
un  backuppc             <none>          <none>          (no description
    ➥ available)
un  baobab               <none>          <node>          (no description
    ➥ available)
un  base                 <none>          <none>          (no description
    ➥ available)
un  base-config          <none>          <none>          (no description
    ➥ available)
ii  base-files           11              amd64           Debian base system
    ➥ miscellaneous files
ii  base-passwd          3.5.46          amd64           Debian base system
    ➥ master password and group files
ii  bash                 5.0-4           amd64           GNU Bourne Again SHell
[..]
$ dpkg -c /var/cache/apt/archives/gnupg-utils_2.2.12-1_amd64.deb
drwxr-xr-x root/root         0 2018-12-15 02:17 ./
drwxr-xr-x root/root         0 2018-12-15 02:17 ./usr/
drwxr-xr-x root/root         0 2018-12-15 02:17 ./usr/bin/
-rwxr-xr-x root/root      3516 2018-12-15 02:17 ./usr/bin/gpg-zip
-rwxr-xr-x root/root    866256 2018-12-15 02:17 ./usr/bin/gpgcompose
-rwxr-xr-x root/root     30792 2018-12-15 02:17 ./usr/bin/gpgparsemail
-rwxr-xr-x root/root     84432 2018-12-15 02:17 ./usr/bin/gpgsplit
-rwxr-xr-x root/root    154952 2018-12-15 02:17 ./usr/bin/gpgtar
-rwxr-xr-x root/root    166568 2018-12-15 02:17 ./usr/bin/kbxutil
-rwxr-xr-x root/root      1081 2017-08-28 12:22 ./usr/bin/lspgpot
-rwxr-xr-x root/root      2194 2018-11-18 23:37 ./usr/bin/migrate-pubring-from-
    ➥ classic-gpg
-rwxr-xr-x root/root    121576 2018-12-15 02:17 ./usr/bin/symcryptrun
-rwxr-xr-x root/root     18424 2018-12-15 02:17 ./usr/bin/watchgnupg
drwxr-xr-x root/root         0 2018-12-15 02:17 ./usr/sbin/
-rwxr-xr-x root/root      3075 2018-12-15 02:17 ./usr/sbin/addgnupghome
-rwxr-xr-x root/root      2217 2018-12-15 02:17 ./usr/sbin/applygnupgdefaults
drwxr-xr-x root/root         0 2018-12-15 02:17 ./usr/share/
drwxr-xr-x root/root         0 2018-12-15 02:17 ./usr/share/doc/
[...]
$ dpkg -I /var/cache/apt/archives/gnupg-utils_2.2.12-1_amd64.deb
 new Debian package, version 2.0.
 size 857408 bytes: control archive=1844 bytes.
    1564 bytes,    32 lines      control
    1804 bytes,    28 lines      md5sums
 Package: gnupg-utils
 Source: gnupg2
 Version: 2.2.12-1
 Architecture: amd64
 Maintainer: Debian GnuPG Maintainers <pkg-gnupg-maint@lists.alioth.debian.org>
```

```
 Installed-Size: 1845
 Depends: libassuan0 (>= 2.0.1), libbz2-1.0, libc6 (>= 2.25), libgcrypt20 (>=
     ➥ 1.8.0), libgpg-error0 (>= 1.26-2~), libksba8 (>= 1.3.4), libreadline7 (>=
     ➥  6.0), zlib1g (>= 1:1.1.4)
 Recommends: gpg, gpg-agent, gpgconf, gpgsm
 Breaks: gnupg (<< 2.1.21-4), gnupg-agent (<< 2.1.21-4)
 Replaces: gnupg (<< 2.1.21-4), gnupg-agent (<< 2.1.21-4)
 Section: utils
 Priority: optional
 Multi-Arch: foreign
 Homepage: https://www.gnupg.org/
 Description: GNU privacy guard - utility programs
  GnuPG is GNU's tool for secure communication and data storage.
  .
  This package contains several useful utilities for manipulating
  OpenPGP data and other related cryptographic elements.  It includes:
  .
   * addgnupghome -- create .gnupg home directories
   * applygnupgdefaults -- run gpgconf --apply-defaults for all users
   * gpgcompose -- an experimental tool for constructing arbitrary
                   sequences of OpenPGP packets (e.g. for testing)
   * gpgparsemail -- parse an e-mail message into annotated format
   * gpgsplit -- split a sequence of OpenPGP packets into files
   * gpgtar -- encrypt or sign files in an archive
   * kbxutil -- list, export, import Keybox data
   * lspgpot -- convert PGP ownertrust values to GnuPG
   * migrate-pubring-from-classic-gpg -- use only "modern" formats
   * symcryptrun -- use simple symmetric encryption tool in GnuPG framework
   * watchgnupg -- watch socket-based logs
[..]
```

GOING FURTHER

Comparison of versions

Since `dpkg` is the program for handling Debian packages, it also provides the reference implementation of the logic of comparing version numbers. This is why it has a `--compare-versions` option, usable by external programs (especially configuration scripts executed by `dpkg` itself). This option requires three parameters: a version number, a comparison operator, and a second version number. The different possible operators are `lt` (strictly less than), `le` (less than or equal to), `eq` (equal), `ne` (not equal), `ge` (greater than or equal to), and `gt` (strictly greater than). If the comparison is correct, `dpkg` returns 0 (success); if not, it gives a non-zero return value (indicating failure).

```
$ dpkg --compare-versions 1.2-3 gt 1.1-4
$ echo $?
0
$ dpkg --compare-versions 1.2-3 lt 1.1-4
$ echo $?
1
$ dpkg --compare-versions 2.6.0pre3-1 lt 2.6.0-1
$ echo $?
```

```
1
```

Note the unexpected failure of the last comparison: for dpkg, `pre`, usually denoting a pre-release, has no particular meaning, and this program compares the alphabetic characters in the same way as the numbers (a < b < c ...), in alphabetical order. This is why it considers "`0pre3`" to be greater than "`0`". When we want a package's version number to indicate that it is a pre-release, we use the tilde character, "~":

```
$ dpkg --compare-versions 2.6.0~pre3-1 lt 2.6.0-1
$ echo $?
0
```

5.4.4. dpkg's Log File

`dpkg` keeps a log of all of its actions in `/var/log/dpkg.log`. This log is extremely verbose, since it details every one of the stages through which packages handled by `dpkg` go. In addition to offering a way to track dpkg's behavior, it helps, above all, to keep a history of the development of the system: one can find the exact moment when each package has been installed or updated, and this information can be extremely useful in understanding a recent change in behavior. Additionally, all versions being recorded, it is easy to cross-check the information with the `changelog.Debian.gz` for packages in question, or even with online bug reports.

5.4.5. Multi-Arch Support

All Debian packages have an Architecture field in their control information. This field can contain either "all" (for packages that are architecture independent) or the name of the architecture that it targets (like "amd64", "armhf", ...). In the latter case, by default, `dpkg` will only accept to install the package if its architecture matches the host's architecture as returned by `dpkg --print-architecture`.

This restriction ensures that users do not end up with binaries compiled for an incorrect architecture. Everything would be perfect except that (some) computers can run binaries for multiple architectures, either natively (an "amd64" system can run "i386" binaries) or through emulators.

Enabling Multi-Arch

`dpkg`'s multi-arch support allows users to define "foreign architectures" that can be installed on the current system. This is simply done with `dpkg --add-architecture` like in the example below. There is a corresponding `dpkg --remove-architecture` to drop support of a foreign architecture, but it can only be used when no packages of this architecture remain.

```
# dpkg --print-architecture
amd64
```

```
# dpkg --print-foreign-architectures
# dpkg -i gcc-8-base_8.3.0-6_armhf.deb
dpkg: error processing archive gcc-8-base_8.3.0-6_armhf.deb (--install):
 package architecture (armhf) does not match system (amd64)
Errors were encountered while processing:
 gcc-8-base_8.3.0-6_armhf.deb
# dpkg --add-architecture armhf
# dpkg --add-architecture armel
# dpkg --print-foreign-architectures
armhf
armel
# dpkg -i gcc-8-base_8.3.0-6_armhf.deb
(Reading database ... 14319 files and directories currently installed.)
Preparing to unpack gcc-8-base_8.3.0-6_armhf.deb ...
Unpacking gcc-8-base:armhf (8.3.0-6) ...
Setting up gcc-8-base:armhf (8.3.0-6) ...
# dpkg --remove-architecture armhf
dpkg: error: cannot remove architecture 'armhf' currently in use by the database
# dpkg --remove-architecture armel
# dpkg --print-foreign-architectures
armhf
```

NOTE

APT's multi-arch support

APT will automatically detect when dpkg has been configured to support foreign architectures and will start downloading the corresponding Packages files during its update process.

Foreign packages can then be installed with `apt install` *`package`*`:`*`architecture`*.

IN PRACTICE

Using proprietary i386 binaries on amd64

There are multiple use cases for multi-arch, but the most popular ones are the possibility to execute (sometimes proprietary) 32 bit binaries (i386) on 64 bit systems (amd64), and the possibility to cross-compile software for a platform or an architecture different from the host one.

Multi-Arch Related Changes

To make multi-arch actually useful and usable, libraries had to be repackaged and moved to an architecture-specific directory so that multiple copies (targeting different architectures) can be installed alongside. Such updated packages contain the "Multi-Arch: same" header field to tell the packaging system that the various architectures of the package can be safely co-installed (and that those packages can only satisfy dependencies of packages of the same architecture). The most important libraries have been converted since the introduction of multi-arch in Debian 7 *Wheezy*, but there are many libraries that will likely never be converted unless someone specifically requests it (through a bug report for example).

```
$ dpkg -s gcc-8-base
```

```
dpkg-query: error: --status needs a valid package name but 'gcc-8-base' is not:
    ➥ ambiguous package name 'gcc-8-base' with more than one installed instance

Use --help for help about querying packages.
$ dpkg -s gcc-8-base:amd64 gcc-8-base:armhf | grep ^Multi
Multi-Arch: same
Multi-Arch: same
$ dpkg -L libgcc1:amd64 |grep .so
/lib/x86_64-linux-gnu/libgcc_s.so.1
$ dpkg -S /usr/share/doc/gcc-8-base/copyright
gcc-8-base:amd64, gcc-8-base:armhf: /usr/share/doc/gcc-8-base/copyright
```

It is worth noting that Multi-Arch: same packages must have their names qualified with their architecture to be unambiguously identifiable. They also have the possibility to share files with other instances of the same package; `dpkg` ensures that all packages have bit-for-bit identical files when they are shared. Last but not least, all instances of a package must have the same version. They must thus be upgraded together.

Multi-Arch support also brings some interesting challenges in the way dependencies are handled. Satisfying a dependency requires either a package marked "Multi-Arch: foreign" or a package whose architecture matches the one of the package declaring the dependency (in this dependency resolution process, architecture-independent packages are assumed to be of the same architecture than the host). A dependency can also be weakened to allow any architecture to fulfill it, with the *package*:any syntax, but foreign packages can only satisfy such a dependency if they are marked "Multi-Arch: allowed".

5.5. Coexistence with Other Packaging Systems

Debian packages are not the only software packages used in the free software world. The main competitor is the RPM format of the Red Hat Linux distribution and its many derivatives. Red Hat is a very popular, commercial distribution. It is thus common for software provided by third parties to be offered as RPM packages rather than Debian.

In this case, you should know that the program `rpm`, which handles RPM packages, is available as a Debian package, so it is possible to use this package format on Debian. Care should be taken, however, to limit these manipulations to extract the information from a package or to verify its integrity. It is, in truth, unreasonable to use `rpm` to install an RPM on a Debian system; RPM uses its own database, separate from those of native software (such as `dpkg`). This is why it is not possible to ensure a stable coexistence of two packaging systems.

On the other hand, the *alien* utility can convert RPM packages into Debian packages, and vice versa.

COMMUNITY

Encouraging the adoption of `.deb`

If you regularly use the `alien` program to install RPM packages coming from one of your providers, do not hesitate to write to them and amicably express your strong

preference for the `.deb` format. Note that the format of the package is not everything: a `.deb` package built with `alien` or prepared for a version of Debian different than that which you use, or even for a derivative distribution like Ubuntu, would probably not offer the same level of quality and integration as a package specifically developed for Debian *Buster*.

```
$ fakeroot alien --to-deb phpMyAdmin-4.7.5-2.fc28.noarch.rpm
phpmyadmin_4.7.5-3_all.deb generated
$ ls -s phpmyadmin_4.7.5-3_all.deb
  4356 phpmyadmin_4.7.5-3_all.deb
```

You will find that this process is extremely simple. You must know, however, that the package generated does not have any dependency information, since the dependencies in the two packaging formats don't have systematic correspondence. The administrator must thus manually ensure that the converted package will function correctly, and this is why Debian packages thus generated should be avoided as much as possible. Fortunately, Debian has the largest collection of software packages of all distributions, and it is likely that whatever you seek is already in there.

Looking at the man page for the `alien` command, you will also note that this program handles other packaging formats, especially the one used by the Slackware distribution (it is made of a simple `tar.gz` archive).

The stability of the software deployed using the `dpkg` tool contributes to Debian's fame. The APT suite of tools, described in the following chapter, preserves this advantage, while relieving the administrator from managing the status of packages, a necessary but difficult task.



Keywords

apt
apt-get
apt-cache
aptitude
synaptic
sources.list
apt-cdrom



Chapter

6

Maintenance and Updates: The APT Tools

Contents

Filling in the `sources.list` File 108 `aptitude`, `apt-get`, and `apt` Commands 116
The `apt-cache` Command 126 The `apt-file` Command 128 Frontends: `aptitude`, `synaptic` 128
Checking Package Authenticity 132 Upgrading from One Stable Distribution to the Next 134
Keeping a System Up to Date 138 Automatic Upgrades 140 Searching for Packages 142

What makes Debian so popular with administrators is how easily software can be installed and how easily the whole system can be updated. This unique advantage is largely due to the APT *program, which Falcot Corp administrators studied with enthusiasm.*

APT is the abbreviation for Advanced Package Tool. What makes this program "advanced" is its approach to packages. It doesn't simply evaluate them individually, but it considers them as a whole and produces the best possible combination of packages depending on what is available and compatible according to dependencies.

VOCABULARY

Package source and source package

The word *source* can be ambiguous. A "source package" — a package containing the source code of a program — should not be confused with a "package source" — a repository (website, FTP server, CD-ROM, local directory, etc.) which contains packages.

APT needs to be given a "list of package sources (repositories)": the file `/etc/apt/sources.list` will list the different repositories that publish Debian packages. APT will then import the list of packages published by each of these sources. This operation is achieved by downloading `Packages.xz` files or a variant such as `Packages.gz` or `.bz2` (using a different compression method) in case of a source of binary packages and by analyzing their contents. In case of a source of source packages, APT downloads `Sources.xz` files or a variant using a different compression method. When an old copy of these files is already present, APT can update it by only downloading the differences (see sidebar "Incremental updates" page 117).

BACK TO BASICS

`gzip`, `bzip2`, `LZMA` and `XZ` Compression

A `.gz` extension refers to a file compressed with the `gzip` utility. `gzip` is the fast and efficient traditional Unix utility to compress files. Newer tools achieve better rates of compression but require more resources (computation time and memory) to compress and uncompress a file. Among them, and by order of appearance, there are `bzip2` (generating files with a `.bz2` extension), `lzma` (generating `.lzma` files) and `xz` (generating `.xz` files).

6.1. Filling in the `sources.list` File

6.1.1. Syntax

Each active line in the `/etc/apt/sources.list` file represents a package source (repository) and is made of at least three parts separated by spaces. For a complete description of the file format and the accepted entry compositions see `sources.list(5)`.

Example 6.1 *Example entry format in `/etc/apt/sources.list`*

```
deb url distribution component1 component2 component3 [..] componentX
deb-src url distribution component1 component2 component3 [..] componentX
```

The first field indicates the source type:

deb package source (repository) of binary packages

deb-src package source (repository) of source packages

The second field gives the base URL of the source. Combined with the filenames listed in the `Packages.xz` files, it must give a full and valid URL. This can consist in a Debian mirror or in any other package archive set up by a third party. The URL can start with file:// to indicate a local source installed in the system's file hierarchy, with http:// or https:// to indicate a source accessible from a web server server, or with ftp:// or ftps:// for a source available on an FTP server. The URL can also start with cdrom: for CD-ROM/DVD/Blu-ray disc based installations, although this is less frequent, since network-based installation methods are eventually more common.

The syntax of the last field depends on the structure of the repository. In the simplest case, you can simply indicate a subdirectory (with a required trailing slash) of the desired source. This is often a simple "`./`" which refers to the absence of a subdirectory. The packages are then directly at the specified URL. But in the most common case, the repositories will be structured like a Debian mirror, with multiple distributions, each having multiple components. In those cases, name the chosen distribution by its "codename" — see the list in sidebar "Bruce Perens, a controversial leader" page 8 — or by the corresponding "suite" oldstable, stable, testing, unstable) and then the components to enable. A typical Debian mirror provides the components main, contrib, and non-free.

VOCABULARY

The `main`, `contrib` and `non-free` archives

Debian uses three components to differentiate packages according to the licenses chosen by the authors of each work. `Main` gathers all packages which fully comply with the Debian Free Software Guidelines[1].

The `non-free` component is different because it contains software which does not (entirely) conform to these principles but which can, nevertheless, be distributed without restrictions. This archive, which is not officially part of Debian, is a service for users who could need some of those programs and, nowadays, also require the firmware for their hardware. However, Debian always recommends giving priority to free software. The existence of this component represents a considerable problem for Richard M. Stallman and keeps the Free Software Foundation from recommending Debian to users.

`Contrib` (contributions) is a set of open source software which cannot function without some non-free elements — these elements can be software from the `non-free` section, or non-free files such as game ROMs, BIOS of consoles, etc. — or some elements, not available from the Debian `main` archive at all. The `contrib` component also includes free software whose compilation requires proprietary elements. This was initially the case for the OpenOffice.org office suite, which used to require a proprietary Java environment.

TIP

Files in `/etc/apt/sources.list.d/`

If many package sources are referenced, it can be useful to split them in multiple files. Each part is then stored in `/etc/apt/sources.list.d/`*`filename`*`.list` (see sidebar "Directories ending in `.d`" page 120).

[1] https://www.debian.org/social_contract.html#guidelines

The cdrom entries describe the CD/DVD-ROMs you have. Contrary to other entries, a CD-ROM is not always available since it has to be inserted into the drive and since only one disc can be read at a time. For those reasons, these sources are managed in a slightly different way, and need to be added with the `apt-cdrom` program, usually executed with the `add` parameter. The latter will then request the disc to be inserted in the drive and will browse its contents looking for `Packages` files. It will use these files to update its database of available packages (this operation is usually done by the `apt update` command). From then on, APT can require the disc to be inserted if it needs one of its packages.

6.1.2. Repositories for *Stable* Users

Here is a standard `sources.list` for a system running the *Stable* version of Debian:

Example 6.2 */etc/apt/sources.list file for users of Debian Stable*

```
# Security updates
deb http://security.debian.org/ buster/updates main contrib non-free
deb-src http://security.debian.org/ buster/updates main contrib non-free

## Debian mirror

# Base repository
deb https://deb.debian.org/debian buster main contrib non-free
deb-src https://deb.debian.org/debian buster main contrib non-free

# Stable updates
deb https://deb.debian.org/debian buster-updates main contrib non-free
deb-src https://deb.debian.org/debian buster-updates main contrib non-free

# Stable backports
deb https://deb.debian.org/debian buster-backports main contrib non-free
deb-src https://deb.debian.org/debian buster-backports main contrib non-free
```

This file lists all sources of packages associated with the *Buster* version of Debian (the current *Stable* suite as of this writing). In the example above, we opted to name "buster" explicitly instead of using the corresponding "stable" aliases (stable, stable-updates, stable-backports) because we don't want to have the underlying distribution changed outside of our control when the next stable release comes out.

Most packages will come from the "base repository" which contains all packages but is seldom updated (about once every 2 months for a "point release"). The other repositories are partial (they do not contain all packages) and can host updates (packages with newer version) that APT might install. The following sections will explain the purpose and the rules governing each of those repositories.

Note that when the desired version of a package is available on several repositories, the first one listed in the `sources.list` file will be used. For this reason, non-official sources are usually added at the end of the file.

As a side note, most of what this section says about *Stable* applies equally well to *Oldstable* since the latter is just an older *Stable* that is maintained in parallel.

Security Updates

Debian takes security seriously. Known software vulnerabilities in Debian are tracked in the Security Bug Tracker[2] and usually get fixed in a reasonable timeframe. The security updates are not hosted on the usual network of Debian mirrors but on security.debian.org, a small set of machines maintained by the Debian System Administrators. This archive contains security updates prepared by the Debian Security Team and/or by package maintainers for the *Stable* and *Oldstable* distribution.

The server can also host security updates for *Testing* but this doesn't happen very often since those updates tend to reach the *Testing* suite via the regular flow of updates coming from *Unstable*.

For serious issues, the security team issues a Debian Security Advisory (DSA) and announces it together with the security update on the debian-security-announce@lists.debian.org mailing list (archive[3]).

Stable Updates

Stable updates are not security sensitive but are deemed important enough to be pushed to users before the next stable point release.

This repository will typically contain fixes for critical and serious bugs which could not be fixed before release or which have been introduced by subsequent updates. Depending on the urgency, it can also contain updates for packages that have to evolve over time, like *spamassassin*'s spam detection rules, *clamav*'s virus database, the daylight-saving time rules of all timezones (*tzdata*), the ESR version of Firefox (*firefox-esr*) or cryptographic keyrings like *debian-archive-keyring*.

In practice, this repository is a subset of the proposed-updates repository, carefully selected by the Stable Release Managers. All updates are announced on the debian-stable-announce@lists.debian.org mailing list (archive[4]) and will be included in the next *Stable* point release anyway.

```
deb https://deb.debian.org/debian buster-updates main contrib non-free
```

[2] https://security-tracker.debian.org
[3] https://lists.debian.org/debian-security-announce/
[4] https://lists.debian.org/debian-stable-announce/

Proposed Updates

Once published, the *Stable* distribution is only updated about once every 2 months. The proposed-updates repository is where the expected updates are prepared (under the supervision of the Stable Release Managers).

The security and stable updates documented in the former sections are always included in this repository, but there is more too, because package maintainers also have the opportunity to fix important bugs that do not deserve an immediate release.

Anyone can use this repository to test those updates before their official publication. The extract below uses the buster-proposed-updates alias which is both more explicit and more consistent since stretch-proposed-updates also exists (for the *Oldstable* updates):

```
deb https://deb.debian.org/debian buster-proposed-updates main contrib non-free
```

Stable Backports

The stable-backports repository hosts "package backports". The term refers to a package of some recent software which has been recompiled for an older distribution, generally for *Stable*.

When the distribution becomes a little dated, numerous software projects have released new versions that are not integrated into the current *Stable* suite, which is only modified to address the most critical problems, such as security issues. Since the *Testing* and *Unstable* suites can be more risky, package maintainers sometimes voluntarily offer recompilations of recent software applications for *Stable*, which has the advantage to users and system administrators to limit potential instability to a small number of chosen packages. The page `https://backports.debian.org` provides more information.

Backports from stable-backports are only created from packages available in *Testing*. This ensures that all installed backports will be upgradable to the corresponding stable version once the next stable release of Debian is available.

Even though this repository provides newer versions of packages, APT will not install them unless you give explicit instructions to do so (or unless you have already done so with a former version of the given backport):

```
$ sudo apt-get install package/buster-backports
$ sudo apt-get install -t buster-backports package
```

6.1.3. Repositories for *Testing*/*Unstable* Users

Here is a standard `sources.list` for a system running the *Testing* or *Unstable* version of Debian:

Example 6.3 */etc/apt/sources.list file for users of Debian* Testing/Unstable

```
# Unstable
deb https://deb.debian.org/debian unstable main contrib non-free
deb-src https://deb.debian.org/debian unstable main contrib non-free

# Testing
deb https://deb.debian.org/debian testing main contrib non-free
deb-src https://deb.debian.org/debian testing main contrib non-free

# Testing security updates
deb http://security.debian.org/ testing-security main contrib non-free
deb-src http://security.debian.org/ testing-security main contrib non-free

# Stable
deb https://deb.debian.org/debian stable main contrib non-free
deb-src https://deb.debian.org/debian stable main contrib non-free

# Stable security updates
deb http://security.debian.org/ stable/updates main contrib non-free
deb-src http://security.debian.org/ stable/updates main contrib non-free
```

NOTE

Layout of security repositories

Starting with Debian 11 *Bullseye*, the codename of the repository providing security updates has been renamed from `codename/updates` into `codename-security` to avoid the confusion with `codename-updates` (see section 6.1.2.2, "Stable Updates" page 111).

With this `sources.list` file APT will install packages from the *Unstable* suite. If that is not desired, use the APT::Default-Release setting (see section 6.2.3, "System Upgrade" page 120) to instruct APT to pick packages from another suite (most likely *Testing* in this case).

There are good reasons to include all those repositories, even though a single one should be enough. *Testing* users will appreciate the possibility to cherry-pick a fixed package from *Unstable* when the version in *Testing* is affected by an annoying bug. On the opposite, *Unstable* users bitten by unexpected regressions have the possibility to downgrade packages to their (supposedly working) *Testing* version.

The inclusion of *Stable* is more debatable but it often gives access to some packages, which have been removed from the development versions. It also ensures that you get the latest updates for packages, which have not been modified since the last stable release.

The Experimental *Repository*

The archive of *Experimental* packages is present on all Debian mirrors, and contains packages which are not in the *Unstable* version yet because of their substandard quality — they are often software development versions or pre-versions (alpha, beta, release candidate...). A package can also be sent there after undergoing subsequent changes which can generate problems. The

maintainer then tries to uncover them with help from advanced users who can handle important issues. After this first stage, the package is moved into *Unstable*, where it reaches a much larger audience and where it will be tested in much more detail.

Experimental is generally used by users who do not mind breaking their system and then repairing it. This distribution gives the possibility to import a package which a user wants to try or use as the need arises. That is exactly how Debian approaches it, since adding it in APT's `sources.list` file does not lead to the systematic use of its packages. The line to be added is:

```
deb https://deb.debian.org/debian experimental main contrib non-free
```

6.1.4. Using Alternate Mirrors

The `sources.list` examples in this chapter refer to package repositories hosted on deb.debian.org[5]. Those URLs will redirect you to servers which are close to you and which are managed by Content Delivery Networks (CDN) whose main role is to store multiple copies of the files across the world, and to deliver them as fast as possible to users. The CDN companies that Debian is working with are Debian partners who are offering their services freely to Debian. While none of those servers are under direct control of Debian, the fact that the whole archive is sealed by GPG signatures makes it a non-issue.

Picky users who are not satisfied with the performance of deb.debian.org can try to find a better mirror in the official mirror list:

➨ `https://www.debian.org/mirror/list`

But when you don't know which mirror is best for you, this list is of not much use. Fortunately for you, Debian maintains DNS entries of the form ftp.*country-code*.debian.org (e.g. ftp.us.debian.org for the USA, ftp.fr.debian.org for France, etc.) which are covering many countries and which are pointing to one (or more) of the best mirrors available within that country.

As an alternative to deb.debian.org, there used to be httpredir.debian.org. This service would identify a mirror close to you (among the list of official mirrors, using GeoIP mainly) and would redirect APT's requests to that mirror. This service has been deprecated due to reliability concerns and now httpredir.debian.org provides the same CDN-based service as deb.debian.org.

6.1.5. Non-Official Resources: mentors.debian.net

There are numerous non-official sources of Debian packages set up by advanced users who have recompiled some software (Ubuntu made this popular with their Personal Package Archive (PPA) service, by programmers who make their creation available to all, and even by Debian developers who offer pre-versions of their package online.

[5] `https://deb.debian.org/`

The mentors.debian.net[6] site is interesting (although it only provides source packages), since it gathers packages created by candidates to the status of official Debian developer or by volunteers who wish to create Debian packages without going through that process of integration. These packages are made available without any guarantee regarding their quality; make sure that you check their origin and integrity and then test them before you consider using them in production.

COMMUNITY

The `debian.net` sites

The *debian.net* domain is not an official resource of the Debian project. Each Debian developer may use that domain name for their own use. These websites can contain unofficial services (sometimes personal sites) hosted on a machine which does not belong to the project and set up by Debian developers, or even prototypes about to be moved on to *debian.org*. Two reasons can explain why some of these prototypes remain on *debian.net*: either no one has made the necessary effort to transform it into an official service (hosted on the *debian.org* domain, and with a certain guarantee of maintenance), or the service is too controversial to be officialized.

Installing a package means giving root rights to its creator, because they decide on the contents of the initialization scripts which are run under that identity. Official Debian packages are created by volunteers who have been co-opted and reviewed and who can seal their packages so that their origin and integrity can be checked.

In general, be wary of a package whose origin you don't know and which isn't hosted on one of the official Debian servers: evaluate the degree to which you can trust the creator, and check the integrity of the package.

GOING FURTHER

Old package versions: `snapshot.debian.org`

The `snapshot.debian.org`[7] service, introduced in April 2010, can be used to "go backwards in time" and to find an old version of a package not longer contained in the Debian archives. It can be used, for example, to identify which version of a package introduced a regression, and more concretely, to come back to the former version while waiting for the regression fix.

6.1.6. Caching Proxy for Debian Packages

When an entire network of machines is configured to use the same remote server to download the same updated packages, any administrator knows that it would be beneficial to have an intermediate proxy acting as a network-local cache (see sidebar "Cache" page 126).

You can configure APT to use a "standard" proxy (see section 6.2.4, "Configuration Options" page 120 for the APT side, and section 11.6, "HTTP/FTP Proxy" page 308 for the proxy side), but the Debian ecosystem offers better options to solve this problem. The dedicated software presented in this section are smarter than a plain proxy cache because they can rely on the

[6] `https://mentors.debian.net`
[7] `https://snapshot.debian.org`